

Autonomous System Project

Ahmed ElMousel*, Zeyad Tahoun*, Hazem Mohamed*, John Karim*, Sarah Bassem*, and Deena Mohey*

*Mechatronics Department - Faculty of Engineering and Materials Science - German University in Cairo, Egypt

Email: john.fahim@student.guc.edu.eg

Abstract—This project targets the implementation of a complete autonomous system that is equipped with the autonomous modules; Control, Path Planning, Navigation, Localization and Mapping. The system which is a 4 wheel differential robot is controlled to reach predefined targets, avoiding collision with the surroundings and other robots while moving towards the targets following a specific trajectory with the use of one of the control algorithms that were developed during the course. The robot is provided with sensors used for collecting information about the robots' states and the surroundings for the localization, mapping, and navigation modules. The hardware was fully implemented a designed and the control module integrating between the hardware and how it should behave using a software code was done on Robot Operating System (ROS) and a simulation of the robot performed using Gazebo software. After completing all the previous steps, the robot successfully completed three trajectories, straight line, circular and compound between both of them.

Index Terms—Autonomous vehicle, ROS, control, path planning, Gazebo, robots.

I. INTRODUCTION

During the last decades, the world mentality was shifting towards autonomous systems, systems that can perform tasks without human direct interface. They consist of both hardware and software, the software is the brain for the human and the hardware translates the commands from software to actions. These systems have levels of autonomy either it is fully autonomous that does not need the aid of humans or perform certain tasks with the help of humans and in this case they are called semi-autonomous or they are fully controllable by humans. The autonomous vehicle is a driver less vehicle that can operate itself without any human interventions also they are called self-driving cars. These cars can sense the surrounding environment through the mounted sensors to get a better understanding of the world.

II. STATE OF ART

Autonomous systems have lots of challenges that's why many researchers published several research work to propose solutions for multiple challenges. A paper published in 2020 by [1] by Queralta et al, reviewed and analysed the existing approaches to multi-robot Search and Rescue (SAR) support, from an algorithmic perspective and putting an emphasis on the methods enabling collaboration among the robots as well as advanced perception through machine vision and multi-agent active perception. Furthermore, they put these algorithms in the context of the different challenges and constraints that various types of robots (ground, aerial, surface or underwater) encounter in different SAR environments (maritime, urban,

wilderness or other post-disaster scenarios). In the same year, a paper published by Naveen et al [2], discussed that a vehicle assistance system was required so not only recognize other vehicles but also provides information on their type and location. The architecture and hardware implementation of a connected vehicle system were described in this project. This unit uses a sensor and GPS system to record data about a moving vehicle's direction and location. It senses the front and rear ends of vehicles and sends data to the on-board vehicle device. The application receives this position info. Collisions with other cars and potential crashes at intersections can be avoided by using this connected vehicle technology. It also aids in providing a traffic-free environment for emergency services.

In 2019, a paper [3] presented a reformulation-framed as a constrained optimization problem-of multi-robot tasks which are encoded through a cost function that is to be minimized. The advantages of this approach are multiple. The constraint-based formulation provides a natural way of enabling long-term robot autonomy applications, where resilience and adaptability to changing environmental conditions are essential. Moreover, under certain assumptions on the cost function, the resulting controller is guaranteed to be decentralized. Furthermore, finite-time convergence can be achieved, while using local information only, and therefore preserving the decentralized nature of the algorithm. The developed control framework has been tested on a team of ground mobile robots implementing long-term environmental monitoring. In [4], they presented a distributed method for formation control of a homogeneous team of aerial or ground mobile robots navigating in environments with static and dynamic obstacles. Their approach leverages both constrained optimization and multi-robot consensus to compute the parameters of the multi-robot formation. This ensures that the robots make progress and avoid collisions with static and moving obstacles. The method allows for reconfiguration. This approach is efficient and scalable with the number of robots. They presented an extensive evaluation of the communication requirements and verify the method in simulations with up to sixteen quadrotors. Lastly, They presented experiments with four real quadrotors flying in formation in an environment with one moving human.

An article conducted in 2018, [5], a new coordination control for several non-holonomic robots competing in target tracking with minimal communications was established. Only the competition winners were given the challenge and were activated to progress towards the goal in this suggested control strategy. The theoretical consistency of a distributed coordination model is shown. A successful technique to tackle

the situation with higher target speed than trackers was also suggested and tested to be extremely effective, inspired by besieging actions of social animals for predation. So, in order to monitor a moving target with several mobile robots in a competitive fashion, a new behaviour has been established in which only the winners of the competition travel towards the target while the others do not. For emergency services, traffic is a significant element that contributes to the time it takes to arrive at the destination. In the case of emergency services such as ambulances, this delay may mean the difference between life and death. Also, in 2018, a paper discussed [6] the various types and architectures of MRS systems and focused on the networking issues, and services that are required to enable MRS systems to be more efficient in performing their roles in their respective applications. They also identified the similarities and differences between mobile ad-hoc networks (MANETs) and MRS systems, analyzed robot-to-robot (R2R) and robot-to-infrastructure (R2I) communication links, and identified the protocols that can be used at the various levels in the MRS hierarchy. In the same year, an article [7] presented a robust and precise localization system that achieves centimeter-level localization accuracy in disparate city scenes and information from complementary sensors such as GNSS, LiDAR, and IMU are used in the system to achieve high localization accuracy and resilience in challenging scenes. An error-state Kalman filter is applied to fuse the localization measurements from different sources with novel uncertainty estimation. the effectiveness of proposed approaches are validated in detail achieving 5-10cm RMS accuracy and outperforming previous state-of-the-art systems.

In 2017, a paper [8] presented safety barrier certificates that ensure scalable and provably collision-free behaviors in multi-robot systems by modifying the nominal controllers to formally satisfy safety constraints. This is achieved by minimizing the difference between the actual and the nominal controllers subject to safety constraints. The resulting computation of the safety controllers is done through a quadratic programming problem that can be solved in real-time. A general framework of minimally invasive collision avoidance for multi-robot systems was formally synthesized using control barrier functions. The computation and sensing requirements were reduced significantly by distributing safety barrier certificates to each individual agents and only considering neighboring agents without losing the safety guarantee. In [9], they presented a speed adjustment method for avoiding obstacles during multi-vehicle cooperative missions. The algorithm assumes knowledge of a nominal trajectory for the obstacle and takes into account bounded deviations in the speed profile. Observing such changes, the algorithm varies the pace of the mission accordingly. Rigorous theoretical results guarantee that the collision is avoided under the cooperative mission scenario.

In an article conducted in 2016 [10], a multi-robot solution was presented for intelligence-gathering tasks in disaster-relief scenarios where communication quality is uncertain. The hardware configuration of two heterogeneous robotic platforms

capable of performing experiments in a relevant field environment and a suite of autonomy enabled behaviors that support operation in a communication-limited setting are described. The results demonstrate autonomy-enabled operation when communication reliability is not sufficient for teleoperation. Furthermore, by allowing the operators to on-the-fly compose behaviors and define sub-missions that respond to new conditions such as navigation failure, safe operation is enabled completely outside the range of reliable communication.

III. METHODOLOGY

A. Hardware

Our robot was first designed as 2 layers on Solidworks software and printed using a CNC Laser cut with acrylic as a material for 2 layers. On the lower layer we put the 4 DC motors, the H-bridge, The Arduino board for low level control and the 2 encoders for the feedback of the motors' position. The upper level layer had the raspberry Pi controller used in high level control, 3 rectangular holes for the holders of the ultrasonic sensors used to identify the surrounding of the robot and a small breadboard containing and inertial measurement unit (IMU) sensor that is used for identifying the heading of the robot. Figure (1), shows the Solidworks design of the car after assembling all components. and Figure (2) shows views for the robot hardware after being assembled in real life.

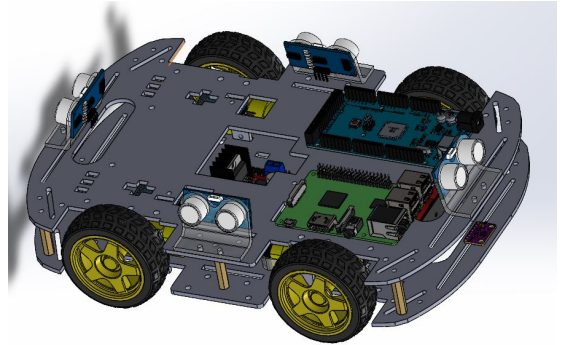


Fig. 1: Solidworks design

B. Controllers

The control of underactuated robots such as husky robot is considered as a hard task in the control theory because the number of inputs are less than the number of degrees of freedom exist in the system. To control such a system, Two different controllers are used. The first one is point to point controller and the second one is Lyapunov controller. Each controller can be described as follows:

1) **Point to Point Controller (P2P)**: It can be described as state feedback controller where linear velocity controlled by the distance between initial and goal positions: $v = k_\rho \rho$ and angular velocity controlled by two angles α and β : $\omega = k_\alpha \alpha + k_\beta \beta$ where ρ , β , and α are the polar coordinates that transformed from x, y, and θ system coordinates to facilitate the controller construction.

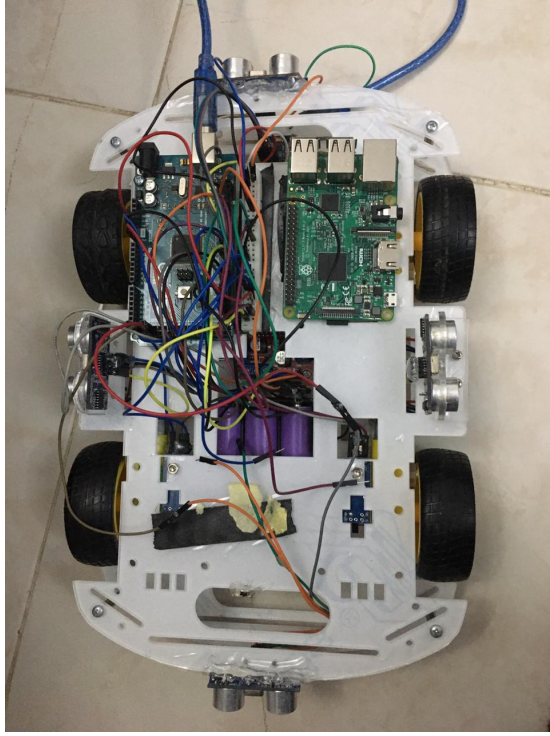


Fig. 2: Assembled robot

2) **Lyapunov Controller:** The idea behind Lyapunov Controller is to check the system stability based on the energy in the system. So Lyapunov control function is selected to describe the energy in the system and the derivative of this function describes the decaying in this energy till reaches to zero. The control law for the linear velocity $v = v_d \cos \theta_r + k_x X_r$, and for the angular velocity $\omega = v_d Y_r + \omega_d + K_\theta \sin \theta_r$.

Figure 3(a) shows the used values for k_ρ , k_α and k_β in P2P Controller and Figure 3(b) shows the gains used in Lyapunov Controller.

K_Rho	K_Alpha	K_Beta
0.5	1.5	-0.6

(a) P2P Gains

K_Rho	K_Beta	Vd	wd
2.5	0.5	1.0	0.5

(b) Lyapunov Gains

Fig. 3: Gains

C. Modeling

The kinematic model of the differential drive mechanism used is as shown in Fig. 4, where x is the position in the x -axis direction, y is the position in the y -axis direction and θ is the orientation. The states are $q = [x, y, \theta]$. For each rotating wheel: the velocity of right wheel is: $V_R = \omega_R R$ Where, ω_R the angular velocity of right wheel and $V_L = \omega_L L$ Where, ω_L the angular velocity of left wheel. Therefore, velocity at center of robot is: $V = \frac{V_R + V_L}{2}$

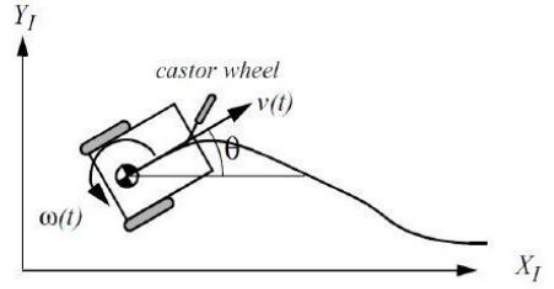


Fig. 4: Kinematic Model for Differential Drive

Studying the states $[\dot{x}, \dot{y}, \dot{\theta}]$, we reach that:

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{R}{2} \cos \theta \\ \frac{R}{2} \sin \theta \\ \frac{R}{L} \end{bmatrix} \dot{\phi}_R + \begin{bmatrix} \frac{R}{2} \cos \theta \\ \frac{R}{2} \sin \theta \\ -\frac{R}{L} \end{bmatrix} \dot{\phi}_L$$

IV. RESULTS

A. Testing the controllers

The 2 controllers have been tested on 8 different scenarios. In each scenario we define the initial position of the robot (X, Y and θ). Also, the desired values for X, Y and θ . These scenarios cover all quadrants, in which the robot can move in its workspace. These scenarios can be shown in the following table. The results show that each controller can reach to the

Scenario No.	X	Y	Theta	X_Des	Y_Des	Theta_Des
1	2	1	0	5	5	90
2	4	1	180	-2	5	90
3	1	-1	0	1	3	180
4	7	-6	90	-6	5	0
5	-1	2	0	9	8	180
6	-3	4	0	-6	6	0
7	-7	-2	0	1	2	90
8	-2	-8	180	-4	2	180

Fig. 5: Testing Scenarios

desired x, y and θ but with error that decreases and increases based on the initial and current positions and the gains used in each controller. Lyapunov controller showed a more stable behavior than the point to point controller. Point to point starts its path with fluctuations in linear and angular velocity till reach to the desired position. Otherwise, Lyapunov starts its path with stable angular and linear velocities, this response can be showed clearly in the response curves. Also, the error in θ increases when the robot rotates counterclockwise in the most scenarios for the 2 implemented controllers. Finally, the main drawback in Lyapunov controller over point to point controller is the reachability to the desired θ so in most scenarios the

error in theta is large when compared the error in θ in point to point controller Figures 3 shows and compares the results for the different tested cases using P2P and Lyapunov controllers respectively: Also, The following graphs show the response curves for the First test scenario using the 2 controllers.

Scenario	Controller	Final Error in X	Final Error in Y	Final Error in θ	RMS Error in X	RMS Error in Y	RMS Error in θ	Max Linear Velocity	Max angular Velocity	Total Distance Traveled	Time to Reach
1	P2P	0.0091	0.4947	0.0896	2.7945	6.7717	0.6982	2.5	1.0	4.7160	6.7369
	Lyapunov	0.5679	0.4923	0.4949	3.9806	5.9892	0.5918	11.86	5.0	5.1966	5.7877
2	P2P	0.0347	0.4999	0.3346	19.0182	4.5382	7.0584	3.56	9.48	10.8457	14.7248
	Lyapunov	0.544	0.3976	1.0461	15.6595	6.2264	2.8695	17.79	2.81	6.5939	7.0692
3	P2P	0.4861	0.0449	0.5846	0.7655	5.8669	3.2978	2.0	1.41	4.6104	6.1191
	Lyapunov	0.5929	0.2709	0.4942	0.5193	3.1646	1.8337	10.01	4.57	5.5485	16.9775
4	P2P	0.4929	0.0409	0.4631	45.9563	54.3036	4.7630	8.52	9.83	22.6602	25.5080
	Lyapunov	0.4061	0.5788	2.4248	57.8871	44.9771	5.8143	40.49	13.0	16.4288	16.8776
5	P2P	0.472	0.0332	0.0734	24.0896	32.1177	6.7004	5.83	0.36	17.1254	20.0504
	Lyapunov	0.5984	0.278	0.4754	14.3015	4.0864	4.6957	27.55	10.03	12.8356	39.0823
6	P2P	0.4826	0.0657	0.7052	2.4851	3.3484	2.8866	1.8	9.33	5.7371	7.7427
	Lyapunov	0.1486	0.4887	1.9855	3.8420	4.0387	3.6785	7.98	3.69	4.2350	4.8876
7	P2P	0.0003	0.4926	0.1617	19.9677	9.5112	1.3361	4.47	0.24	9.2728	11.0713
	Lyapunov	0.3331	0.5982	0.4478	19.8391	3.2707	1.1718	22.18	6.02	9.8155	13.98
8	P2P	0.4916	0.0021	0.2021	5.4798	35.6973	6.2847	4.65	6.54	11.3745	13.9334
	Lyapunov	0.5737	0.2541	0.7030	0.8785	36.1383	3.4082	23.9	8.07	11.2046	13.2412

Fig. 6: Testing Results

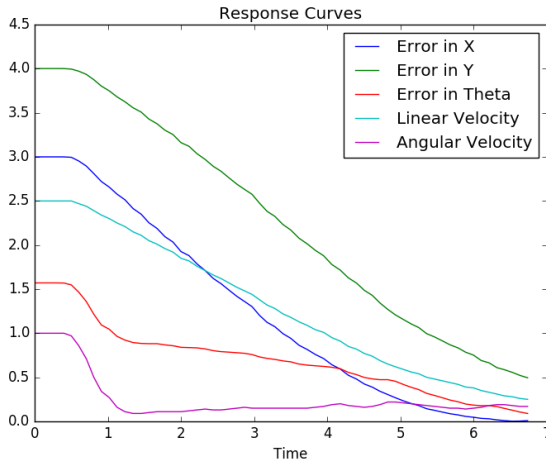


Fig. 7: First case Curve for P2P

B. Open loop Test

C. Serial communication Test

1) *Led Experiments:* In this experiment, we implemented communication protocol between high and low-level processors. where the high-level processor is the Raspberry pi, while the low-level is the Arduino. All codes written on the Raspberry pi are in the form of ROS nodes communicating through publishing and subscribing messages. In this task, the chosen communication protocol was tested on a simple experiment. Two different experiments were designed with two ROS nodes that were launched in the created launch file: In **Experiment 1:** A ROS node on the high-level processor write something and the transmitted data was read by an Arduino script. The transmitted data was shown on the LED. This means that the LED is connected to the Arduino pins. **Experiment 2:** A ROS

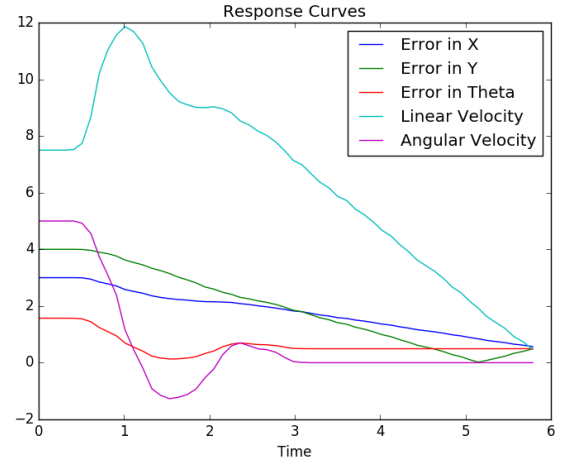


Fig. 8: First Case Curve for Lyapunov

node named on the high-level processor read something. The transmitted data was written by an Arduino script named. The transmitted data were shown on the LED. This means that the LED is connected to the ROS-enabled processor pins.

The results showed success in both experiments.

2) *Sensors Experiment:* This experiment can be considered as an upgrade of the previous two experiments so we collect the sensory data on the low level controller and publishes this data through ROS nodes to the other nodes on the high level controller like planning and control nodes which will be discussed late. Also, Low level controller subscribes the desired speed from the control node to drive the motor. This operation of sending data back and forth between the high and low level controllers is done through a communication node that subscribes the desired speeds from a control node and sends the subscribed desired speeds to the Arduino and receives the sensory data from the Arduino. Then publishes the received data to other nodes; planning and control.

The results showed success in publishing and subscribing the data with some delay in case of odometry data type while publishing the IMU sensor data. Also, it causes overflow buffering problem which means that the message published is larger than the buffer size and this problem can be fixed by editing the output buffer size in ros setup from 512 byte to 1024 byte.

D. Closed loop Test

V. CONCLUSION AND FUTURE RECOMMENDATIONS

After completing the developing of the hardware and preparing it to receive sensory data and accept actuation orders from the low level controller (Arduino) that was calculated using P2P as well as high level controller (Raspberry pi) using Lyapunov control laws, the first few iterations show that the autonomy were the OLR of trajectories such as straight line and circular paths at several percentages of max velocity of the robot. performing some CLR of same trajectories, an actual autonomous system of a mobile robot can be expanded

into many different applications of the autonomous industry, tackling obstacles and changing trajectories while in iterations defining new goal positions. These rewarding results show that it is possible to further expand to many applications needing autonomy like intersection management, autopilot operations, auto-parking.

REFERENCES

- [1] J. P. Queralta, J. Taipalmaa, B. C. Pullinen, V. K. Sarker, T. N. Gia, H. Tenhunen, M. Gabbouj, J. Raitoharju, and T. Westerlund, "Collaborative multi-robot systems for search and rescue: Coordination and perception," *arXiv preprint arXiv:2008.12610*, 2020.
- [2] N. C., "Connected vehicle system for crash avoidance and emergency vehicle," *International Journal for Research in Applied Science and Engineering Technology*, vol. 8, pp. 33–39, 09 2020.
- [3] G. Notomista and M. Egerstedt, "Constraint-driven coordinated control of multi-robot systems," in *2019 American Control Conference (ACC)*. IEEE, 2019, pp. 1990–1996.
- [4] J. Alonso-Mora, E. Montijano, T. Negeli, O. Hilliges, M. Schwager, and D. Rus, "Distributed multi-robot formation control in dynamic environments," *Autonomous Robots*, vol. 43, no. 5, pp. 1079–1100, 2019.
- [5] S. E. Shladover, "Connected and automated vehicle systems: Introduction and overview," *Journal of Intelligent Transportation Systems*, vol. 22, no. 3, pp. 190–200, 2018. [Online]. Available: <https://doi.org/10.1080/15472450.2017.1336053>
- [6] I. Jawhar, N. Mohamed, J. Wu, and J. Al-Jaroodi, "Networking of multi-robot systems: Architectures and requirements," *Journal of Sensor and Actuator Networks*, vol. 7, no. 4, p. 52, 2018.
- [7] G. Wan, X. Yang, R. Cai, H. Li, Y. Zhou, H. Wang, and S. Song, "Robust and precise vehicle localization based on multi-sensor fusion in diverse city scenes," in *2018 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2018, pp. 4670–4677.
- [8] L. Wang, A. D. Ames, and M. Egerstedt, "Safety barrier certificates for collisions-free multirobot systems," *IEEE Transactions on Robotics*, vol. 33, no. 3, pp. 661–674, 2017.
- [9] S. B. Mehdi, V. Cichella, T. Marinho, and N. Hovakimyan, "Collision avoidance in multi-vehicle cooperative missions using speed adjustment," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*. IEEE, 2017, pp. 2152–2157.
- [10] J. Gregory, J. Fink, E. Stump, J. Twigg, J. Rogers, D. Baran, N. Fung, and S. Young, "Application of multi-robot systems to disaster-relief scenarios with limited communication," in *Field and Service Robotics*. Springer, 2016, pp. 639–653.